LES JARDINS DE COCAGNE



PROPOSITION **TECHNIQUE**

FREY Alyssia - ELIK Asya

LES JARDINS DE COCAGNE

SOMMAIRE

1. Cadrage Métier & Exigences

2. Architecture Globale & Persistance

3. Module Logistique & Cartographie — Sujet B

4. Modules Métier : Planification & Recettes — Sujets B et C

5. Industrialisation, Déploiement & Bilan



PARTIE I
CADRAGE MÉTIER & EXIGENCES

LES JARDINS DE COCAGNE

1. CONTEXTE MÉTIER : LE RÉSEAU COCAGNE

Sujet B:

Logistique des tournées:
Conception d'un outil permettant d'organiser des tournées. En fonction du nombre de paniers à livrer, de la capacité des dépôts, de nombre et de la la capacité des camionnettes.

Sujet B:

Carte de livraison :
Concevoir l'interface permettant de créer les tournées de livraison. Sur une carte dessiner l'itinéraire de la tournée en fonction des dépôts sélectionnés pour faire partie de cette livraison.

Sujet C:

Gestion des recettes:
Concevoir une rubrique permettant de gérer des recettes de cuisine. Ajouter un moteur de recherche textuel

Planification calendaire:
Ordonnancement temporel des livraisons et des créneaux de dépôt pour les chauffeurs et gestionnaires.

Modèle socio-économique des Jardins de Cocagne :

- Réseau fédérant près de 100 chantiers d'insertion (ACI) en maraîchage biologique.
- Exploitations associatives indépendantes : absence totale de subordination juridique.
- Accompagnement moyen de 22 salariés en parcours d'insertion par structure.

Activité commerciale & distribution :

- Production de 70 à 80 variétés de légumes distribuées sous forme d'abonnements paniers.
- Distribution en circuit court via des points de dépôt (PDD) publics ou réservés.

Enjeu du projet :

- Fournir une solution mutualisée nationale tout en garantissant l'indépendance de gestion locale.

1.1 DIAGNOSTIC TECHNIQUE DE L'OUTIL EXISTANT

CADRE MÉTIER

Le Réseau Cocagne & Les Jardins

- Près de 100 associations loi 1901 en agriculture biologique et chantiers d'insertion (ACI).
- Moyenne de 22 salariés en insertion par structure (chômage longue durée, réfugiés, précarité).
- 70 à 80 variétés de légumes distribuées sous forme de paniers hebdomadaires ou bimensuels.
- Modèle décentralisé : indépendance juridique mais besoin fort de mutualisation logicielle.

DIAGNOSTIC TECHNIQUE

Limites de la Solution Actuelle

- Base Microsoft Access historique (années 2000) utilisée encore par plus de 60 % des jardins.
- Outil monoposte, rigide et incapable d'interagir nativement avec le Web ou le mobile.
- Absence totale d'optimisation cartographique et de gestion dynamique des flottes de véhicules.
- 75 % des Jardins expriment un besoin critique de modernisation vers un outil SaaS moderne.

Constat sur le progiciel historique :

- Conçu dans les années 2000 sur Microsoft Access, encore utilisé par plus de 60% des jardins.

Facteurs bloquants identifiés :

- Fichier monoposte local : risque permanent de corruption lors des accès concurrents.
- Aucun interfaçage Web/Mobile : gestion des adhérents manuelle et chronophage.
- Absence d'optimisation cartographique : tournées et feuilles de route tracées à la main.

Urgence opérationnelle :

- 75% des structures ont exprimé un besoin immédiat de migration vers une application SaaS moderne.

1.2 EXIGENCES NON FONCTIONNELLES & CONTRAINTES

Accessibilité et ergonomie :

- Prise en compte du public en insertion : interfaces didactiques, infobulles d'aide contextuelle et guidage renforcé.

Performances attendues :

- Temps de réponse transactionnel inférieur à 200 ms.
- Calcul de l'optimisation d'une tournée logistique en moins de 3 secondes.

Conformité réglementaire :

- RGPD : Droit d'accès, de rectification et purge automatique des données adhérents.
- Loi Anti-Fraude & Normes bancaires : Traçabilité inaltérable des encaissements et respect du standard SEPA.

LES JARDINS DE COCAGNE

1.3 MÉTHODOLOGIE : QU'EST-CE QU'UNE FICHE ADR ?

- **Définition** : (Architecture Decision Record) C'est un document court qui capture une décision architecturale significative, son contexte et ses conséquences.

- **Structure** : 1. Contexte & Problème | 2. Alternatives analysées | 3. Décision arbitrée | 4. Conséquences & Risques acceptés.

- **Pourquoi les utiliser :**
- Mémoire technique : éviter l'oubli du « pourquoi » d'un choix technique plusieurs mois ou années plus tard.
- Immuabilité : une fiche validée ne se modifie plus ; si un choix change, un nouvel ADR est créé pour remplacer l'ancien.
- Onboarding accéléré : permettre aux nouveaux arrivants dans l'équipe de comprendre rapidement les arbitrages passés.
- Alignement d'équipe : rendre les décisions transparentes, débattues et documentées sans la lourdeur d'un dossier d'architecture de 50 pages.
- Application : 9 choix architecturaux cardinaux tracés dans la documentation du projet (/documents/fiche_adr/).

Source

PARTIE II
ARCHITECTURE GLOBALE & PERSISTANCE

LES JARDINS DE COCAGNE

2. SCHÉMA D'URBANISATION N-TIERS

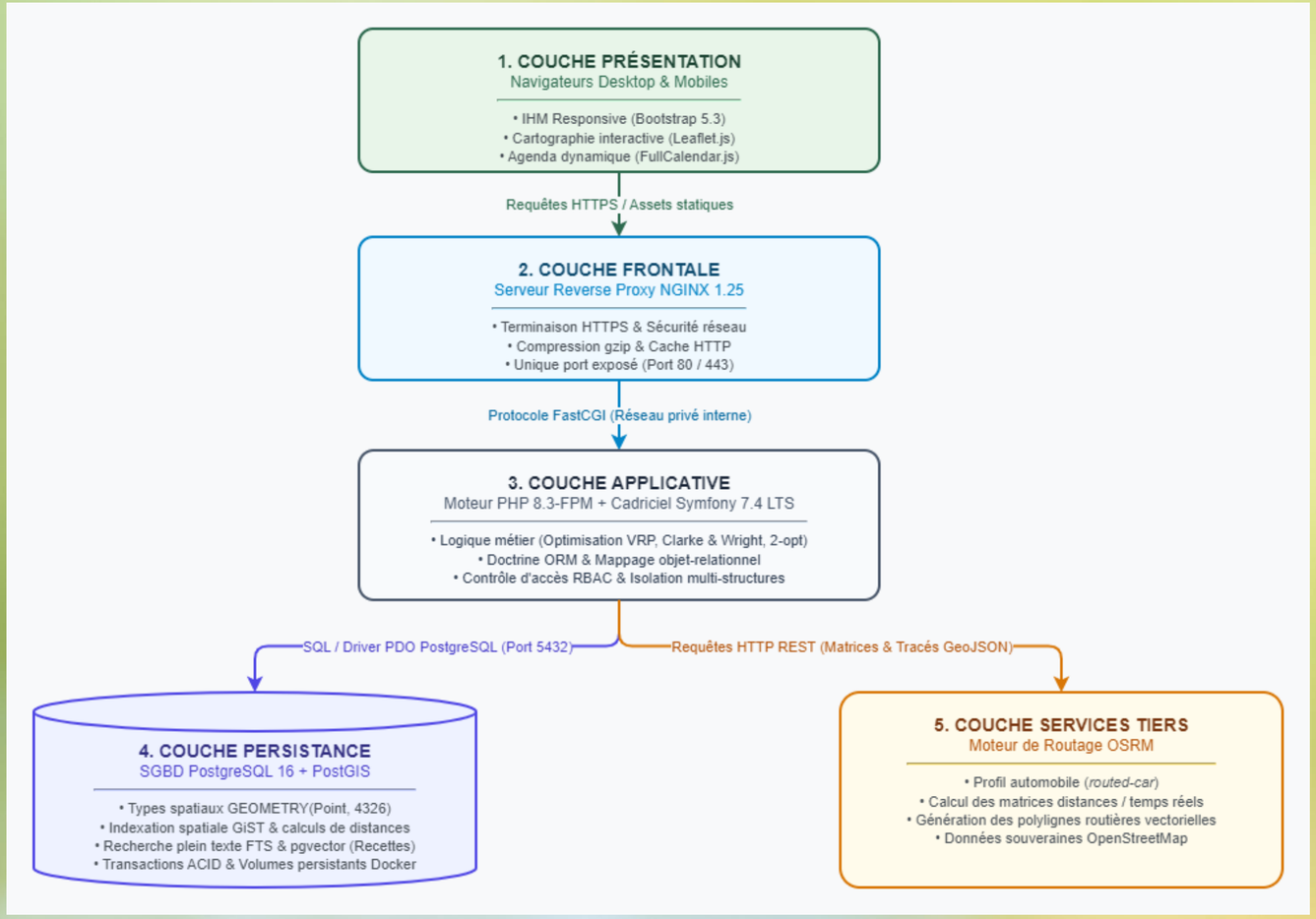
1. Couche Présentation : Navigateurs Desktop et Mobiles (Leaflet.js, FullCalendar.js, Bootstrap 5.3).

2. Couche Frontale : Reverse Proxy NGINX 1.25 (terminaison HTTPS, compression gzip, port public 80/443).

3. Couche Applicative : PHP 8.3-FPM avec Symfony 7.4 LTS (moteur métier, Doctrine ORM, sécurité RBAC).

4. Couche Persistance : PostgreSQL 16 + extensions PostGIS et pgvector (ACID, types spatiaux et sémantiques).

5. Couche Services Tiers : Moteur Open Source Routing Machine (OSRM) pour les calculs d'itinéraires routiers réels.



LES JARDINS DE COCAGNE

ADR 01 : MODÈLE DE RESTITUTION WEB (HYBRIDE VS SPA)

Interface web hybride

Contexte	Restituer une interface réactive intégrant carte et agenda dynamiques.
Alternatives analysées	• **SPA intégrale React/Vite :** coût de maintenance lourd, gestion JWT complexe. • **Serveur Twig pur :** rechargement complet pénalisant la carte.
Décision	**Modèle hybride Twig + Symfony UX (Stimulus & Turbo).**
Conséquences	Rendu serveur sécurisé nativement anti-XSS, micro-contrôleurs JS légers.

Twig est le moteur de templates natif de Symfony et nous le connaissons déjà. Stimulus et Turbo apportent la réactivité de la carte et de l'agenda sans réécrire une application complète en React, donc avec beaucoup moins de code à développer et à maintenir.

LES JARDINS DE COCAGNE

ADR 02 : MOTEUR DE BASE DE DONNÉES RELATIONNELLE ET SPATIALE

Base de données

Contexte	Centraliser les données de gestion, la géolocalisation des dépôts et les contraintes ACID.
Alternatives analysées	• **MySQL 8.0 :** fonctions spatiales partielles, indexation FTS française limitée. • **MongoDB :** NoSQL inadapté aux règles comptables et quotas.
Décision	**PostgreSQL 16 avec extension spatiale PostGIS.**
Conséquences	Requêtage spatial standardisé (SRID 4326), robustesse transactionnelle, base unique pour tous les besoins.

PostgreSQL s'intègre directement avec Doctrine, donc on garde la même façon de travailler que d'habitude. PostGIS couvre la géolocalisation des dépôts dans la même base, sans second outil.

LES JARDINS DE COCAGNE

ADR 03 : CADRICIEL BACKEND APPLICATIF (SYMFONY VS CONCURRENTS)

Choix du framework serveur

Contexte	Socle serveur pour piloter les règles métier, la sécurité et les traitements batch.
Alternatives analysées	• **Laravel 11 :** prise en main rapide, mais ORM Eloquent (Active Record) couplant trop fortement le modèle à la base. • **NestJS / Node.js :** asynchronisme performant, mais ORMs (Prisma, TypeORM) moins éprouvés sur les verrous pessimistes et PostGIS. • **Django (Python) :** très robuste pour la data, mais éloigné de la stack et des compétences de l'équipe.
Décision	**Symfony 7.4 LTS**
Conséquences	ORM Doctrine en Data Mapper pour découpler le domaine, composant Messenger natif pour l'asynchrone, support long terme de 3 ans garanti.

On a choisi Symfony parce que l'équipe le maîtrise déjà, ce qui nous évite une phase d'apprentissage et nous fait gagner du temps pour livrer les fonctionnalités. Il répond en plus aux besoins du projet, avec Doctrine pour découpler le domaine, Messenger pour l'asynchrone et un support long terme de trois ans.

LES JARDINS DE COCAGNE

ADR 04 : STRATÉGIE D'ISOLATION MULTI-STRUCTURES

Multi-tenance

Contexte	Faire cohabiter 100 associations indépendantes sur une même instance sans fuite de données.
Alternatives analysées	• **Base de données par structure :** coût d'infrastructure et migrations ingérables. • **Schéma PostgreSQL par structure :** complexité de pooling de connexions.
Décision	**Base unique partagée avec clé de partitionnement structure_id et Filtre Doctrine SQL global.**
Conséquences	Injection automatique et transparente de la clause WHERE structure_id = :id sur 100 % des requêtes applicatives.

Le filtre SQL de Doctrine est un mécanisme que l'on utilise déjà dans l'écosystème Symfony, et il applique automatiquement le structure_id à toutes les requêtes. On garde ainsi une seule base à maintenir, au lieu de 100 bases ou schémas à migrer séparément.

LES JARDINS DE COCAGNE

ADR 05 : TRAITEMENT ASYNCHRONE DES CALCULS ET ÉDITIONS

ADR 05 – Traitements asynchrones

Contexte	Le calcul de tournées et la génération massive de PDF (feuilles de route) ne doivent jamais bloquer l'IHM web. Les workers asynchrones génèrent en vectoriel les trois documents obligatoires des chantiers d'insertion : La feuille de préparation (récapitulatif global des légumes et paniers à confectionner le matin), Les étiquettes paniers nominatives (collées sur les cagettes, avec le nom de l'adhérent, la formule et le dépôt) La feuille de route du chauffeur (ordre des dépôts, contacts sur place, case d'émargement).
Alternatives analysées	Traitement synchrone HTTP : risque d'erreur HTTP 504. Broker RabbitMQ ou Redis dédié : serveur et mémoire supplémentaires requis.
Décision	Symfony Messenger adossé à une table de file d'attente PostgreSQL.
Conséquences	Acquittement HTTP immédiat (202 Accepted) : La demande part en file d'attente et l'IHM reste disponible. Un worker d'arrière-plan, managé en conteneur, traite la file. Aucun service externe n'est requis, la file étant une table de la base PostgreSQL existante.

Messenger est déjà intégré à Symfony, donc on n'ajoute ni nouvel outil à apprendre ni nouveau serveur à installer. Le seul élément à mettre en place est un worker, et la file d'attente reste dans PostgreSQL, ce qui simplifie le déploiement.

LES JARDINS DE COCAGNE

2.1 MODÈLE D'HABILITATION & CONTRÔLE D'ACCÈS (RBAC)

Rôle Applicatif	Périmètre Autorisé	Actions Métier Permises
ROLE_CLIENT	Espace Adhérent privé	Inscription, choix du dépôt, consultation du calendrier et recettes.
ROLE_LIVREUR	Feuille de route mobile / Interface mobile	Consultation de la feuille de route et validation des émargements.
ROLE_ENCADRANT	Back-office du Jardin	Gestion des tournées, dépôts, composition des paniers et adhérents.
ROLE_SUPER_ADMIN	Supervision Réseau	Création de structures et consultation des métriques nationales.

Utilisation des **Voters Symfony** pour valider la propriété d’une ressource avant modification.

LES JARDINS DE COCAGNE

2.2 MODÈLE PHYSIQUE CIBLÉ : LOGISTIQUE & DÉPÔTS

Relations entre tables clés du périmètre :

Entités clés du périmètre :

- DEPOT : id (UUID), structure_id, nom, capacite_max, position (Point GPS), categorie, code_acces.
- TOURNEE : id (UUID), structure_id, vehicule_id, code_tournee, date_livraison, statut.
- ARRET_TOURNEE : id, tournee_id, depot_id, ordre_passage, heure_estimee, nb_paniers.
- VEHICULE : id, structure_id, immatriculation, capacite_paniers, charge_utile_kg.

Contrainte d'intégrité : Clé d'unicité composite sur (tournee_id, ordre_passage) interdisant les doublons de passage

LES JARDINS DE COCAGNE

2.3 STOCKAGE & INDEXATION SPATIALE POSTGIS

Définition de la colonne géométrique :
• Coordonnées GPS stockées sous le standard WGS 84 (SRID 4326) : GEOMETRY(Point, 4326).

Indexation haute performance :
• Index GiST spatial : CREATE INDEX idx_depot_geom ON depot USING GIST (position);

Requête spatiale de proximité :
- Détection instantanée des dépôts situés dans un rayon de 10 km autour du domicile d'un adhérent via la fonction ST_DWithin

LES JARDINS DE COCAGNE

2.4 GESTION DES CAPACITÉS & VERROU CONCURRENCE

Problématique : Empêcher le dépassement de capacité d'un dépôt lors de validations simultanées d'abonnements en ligne.

Stratégie de verrouillage pessimiste :

- Encapsulation dans une transaction Doctrine avec LockMode::PESSIMISTIC_WRITE.
- Requête SQL sous-jacente : SELECT * FROM depot WHERE id = :id FOR UPDATE.
- La ligne est verrouillée le temps de recalculer les inscriptions actives ; tout dépassement de capacite_max déclenche une exception métier bloquante.

PARTIE III
MODULE LOGISTIQUE & CARTOGRAPHIE

LES JARDINS DE COCAGNE

3 FORMALISATION DU PROBLÈME : MODÈLE CVRP

Nature du problème logistique :

- Problème de tournées de véhicules avec capacités limitées.

Contraintes opérationnelles simultanées :

- Flotte hétérogène de véhicules avec limite de charge stricte.
- Capacité d'accueil maximale de chaque point de dépôt.
- Respect des fenêtres horaires de dépôt imposées par les commerces et partenaires.

Réduire la distance parcourue, le temps de trajet et l'empreinte carbone globale

3.1 HEURISTIQUE CONSTRUCTIVE : ALGORITHME DE CLARKE & WRIGHT

Il faut répondre à la question « Quels dépôts regrouper ensemble dans la même camionnette ? »

Fonctionnement mathématique des gains (savings) :

On part de la pire solution : un aller-retour dédié depuis le Jardin vers chaque dépôt individuel. Pour chaque paire de dépôts (i, j), on calcule le gain kilométrique réalisé en les fusionnant :

$$s(i, j) = \text{Distance}(i, \text{Jardin}) + \text{Distance}\}(\text{Jardin}, j) - \text{Distance}(i, j)$$

Les gains sont triés par ordre décroissant et on fusionne les dépôts en commençant par les gains les plus rentables, tant que la somme des paniers ne dépasse pas la capacité du véhicule.

LES JARDINS DE COCAGNE

3.2 OPTIMISATION LOCALE : ALGORITHME 2-OPT

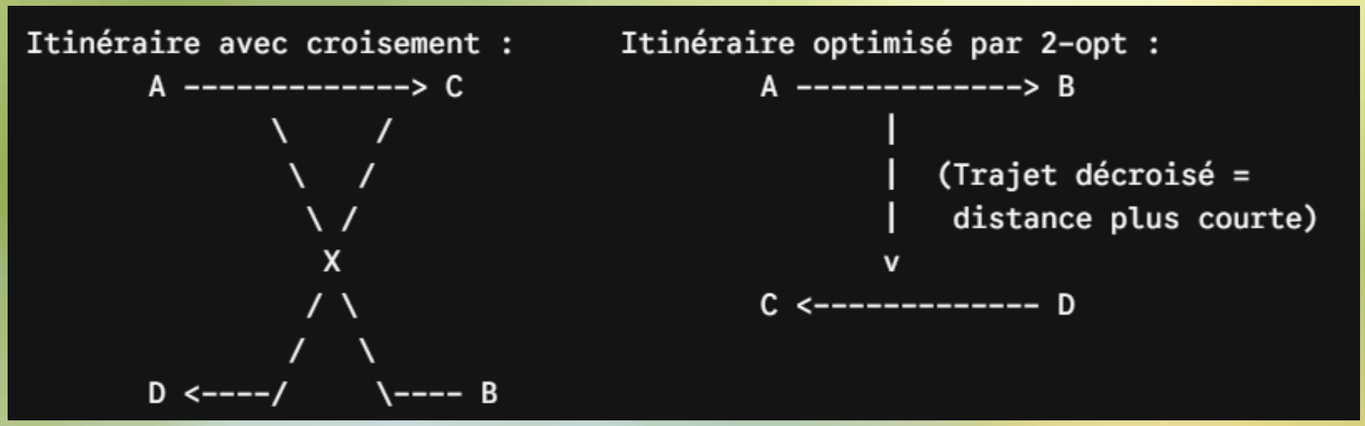
Post-traitement géométrique des tournées calculées :

- Il faut éliminer les croisements de routes sur une tournée faite.
- Décroisement itératif de deux dépôts (A-C et B-D) si la liaison A-B et C-D est plus courte.

Impact mesuré :

- Une réduction moyenne de 8 à 14% sur le kilométrage global des livreurs.
- Remet les dépôts dans le meilleur ordre possible pour éliminer les croisement et réduire l'essence consommée.

Une fois que Clarke & Wright a choisi les dépôts à mettre dans la camionnette, 2-opt intervient pour répondre à la question :
« Dans quel ordre précis doit-on les visiter pour faire le moins de kilomètres possible ? »



LES JARDINS DE COCAGNE

ADR 06 : BIBLIOTHÈQUE DE CARTOGRAPHIE INTERACTIVE

Cartographie web

Contexte	Afficher les points de dépôts et visualiser les boucles logistiques sur une carte web.
Alternatives analysées	Google Maps Platform : coûts récurrents d'API, tracking utilisateur. Mapbox GL : licence commerciale restrictive.
Décision	Leaflet.js adossé aux tuiles OpenStreetMap (OSM).
Conséquences	Bibliothèque ultra-légère (42 Ko), totalement gratuite, souveraine et conforme à l'éthique Cocagne.

Leaflet s’intègre simplement avec Stimulus, donc on le branche dans notre interface Twig sans rien ajouter à la stack. Il est gratuit et sans tracking, ce qui colle aux valeurs de Cocagne, et il couvre ce dont on a besoin : afficher les dépôts et les tournées.

LES JARDINS DE COCAGNE

ADR 07 : CALCUL D'ITINÉRAIRES ET MÉTRIQUES ROUTIÈRES

Calcul d'itinéraires

Contexte	Calculer la distance et le temps de conduite réel entre les dépôts à travers le relief vosgien.
Alternatives analysées	Calcul à vol d'oiseau euclidien : faussé par les cols et vallées. API GraphHopper externe : quotas stricts et facturation.
Décision	Moteur OSRM (Open Source Routing Machine) auto-hébergé.
Conséquences	Calculs de matrices distances/temps en quelques millisecondes, polylignes GeoJSON routières précises, zéro dépendance à un service tiers payant.

OSRM tourne dans un conteneur Docker, comme le reste de notre stack, donc on le déploie avec un simple docker compose up -d sans nouvel outil à apprendre. Il utilise les données OpenStreetMap, déjà utilisées pour la carte Leaflet, et il donne un temps de conduite réaliste sans dépendre d’une API payante.

3.3 CONTRÔLEUR CARTOGRAPHIQUE STIMULUS

Le Stimulus.js permet un dynamisme cartographique en temps réel sans rupture de navigation grâce :

- Pilotage réactif de la carte sans recharger la page.
- Cycle de vie : À l'affichage de la tournée, appel asynchrone fetch('/api/tournees/{id}/depots').
- Affichage cartographique : Centrage dynamique de Leaflet sur les coordonnées des dépôts et distinction visuelle des marqueurs selon leur catégorie.
- Tracé : Réception du flux GeoJSON fourni par OSRM et rendu vectoriel via L.geoJSON().

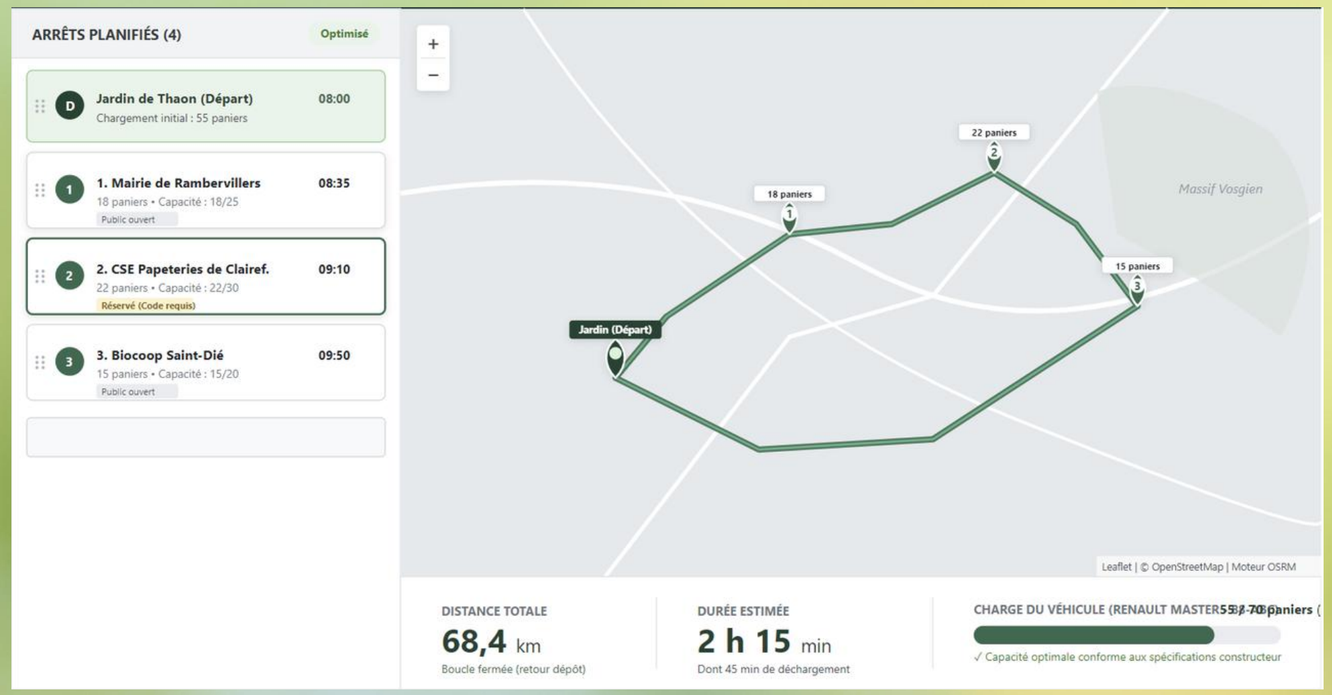
LES JARDINS DE COCAGNE

3.4 SYNOPTIQUE DE L'INTERFACE DE LIVRAISON

Agencement ergonomique de l'écran gestionnaire :

- À gauche : Liste ordonnée des arrêts avec poignées de glisser-déposer.
- Au centre : Carte Leaflet avec tracé OSRM et marqueurs numérotés.
- En bas : Bandeau des indicateurs clés (Distance totale 68,4 km, durée estimée 2 h 15, jauge de remplissage véhicule à 78 %).

POTENTIEL VISUEL :



PARTIE IV
MODULES MÉTIER : PLANIFICATION & RECETTES (SUJETS B ET C)

LES JARDINS DE COCAGNE

4. COMPOSANT CALENDRIER FULLCALENDAR.JS

Visualisation et manipulation ergonomique du calendrier de livraison :

Options évaluées :

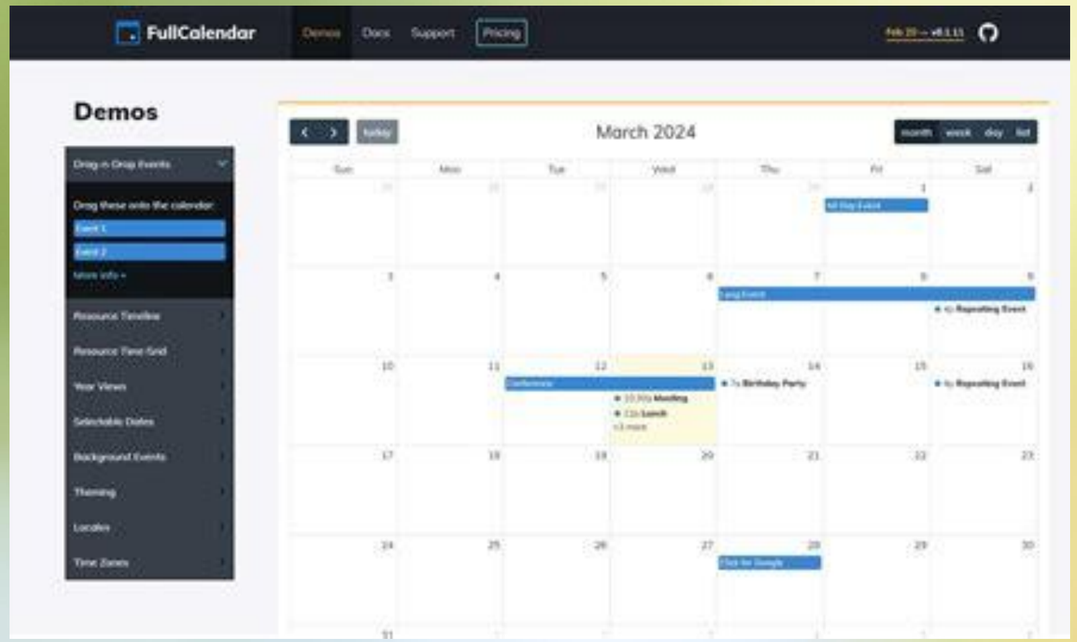
- Agenda maison développé de zéro : Temps de développement excessif et risque de bugs.
- FullCalendar.js : Standard open source de référence, support du glisser-déposer et vues multiples.

Option choisi

Calendrier Interactif: FullCalendar.js

Planification visuelle:
Vues mois, semaine et jour pour visualiser les tournées de chaque camionnette.

Glisser-déposer (Drag & Drop):
Reprogrammation intuitive reliée à un endpoint Symfony pour vérifier les contraintes de disponibilité des dépôts.



LES JARDINS DE COCAGNE

4.1 ALGORITHME DE GÉNÉRATION DES DISTRIBUTIONS

Calcul du plan d'approvisionnement saisonnier :

• Prise en compte de la date de démarrage et de fin de saison associative.

• Fréquence de distribution paramétrable : hebdomadaire, tous les 15 jours (semaines paires/impaires).

• Exclusion stricte des semaines de fermeture annuelle déclarées par la structure maraîchère.

• Vérification d'intégrité : placement strict du quota contractuel de paniers prévus.

LES JARDINS DE COCAGNE

4.2 GESTION DYNAMIQUE DES JOURS FÉRIÉS

Interfaçage avec l'API publique des jours fériés français :

• Intégration de **calendrier.api.gouv.fr** pour l'acquisition automatique des fériés nationaux.

Règles métier de décalage des tournées :

• Vendredi férié : Décalage obligatoire au Jeudi précédent (pas de report le samedi).
• Lundi férié : Décalage obligatoire au Mardi suivant (pas de report le dimanche).
• Synchronisation : La date de préparation en atelier maraîcher est décalée en conséquence.

LES JARDINS DE COCAGNE

4.3 OBJECTIFS FONCTIONNELS DU MODULE RECETTES

Enjeu pour les Jardins de Cocagne :

- Guider les adhérents dans la valorisation des légumes méconnus ou livrés en abondance dans le panier (panais, blettes, rutabagas).
- Proposer automatiquement des recettes alternatives dès qu'un légume est en rupture ou qu'un adhérent consulte un plat similaire.

Mutualisation des structures:

Base de données partagée : chaque recette rédigée par un encadrant ou un bénévole d'un Jardin local alimente toutes les structures.

Trouver les recettes qui se ressemblent :

- La recherche dans les bars de recherche mot a mot, ne convient pas à comprendre la ressemblance culinaire. Par exemple : une « potée aux choux » et un « mijoté de légumes » n'ont pas les mêmes mots, mais se préparent de la même façon.
- Il faut donc modéliser les recettes sous forme de vecteurs sémantiques pour calculer leur degré de ressemblance mathématique.

LES JARDINS DE COCAGNE

ADR 08 : MOTEUR DE RECHERCHE TEXTUELLE DES RECETTES

Recherche de recettes

Contexte	Permettre aux adhérents de retrouver des recettes par titre, ingrédients et étapes de préparation.
Alternatives analysées	Recherche SQL basique LIKE '%mot%' : lente, sensible aux fautes, sans pertinence. Cluster Elasticsearch dédié : consommation mémoire excessive (> 2 Go de RAM).
Décision	PostgreSQL Full-Text Search (FTS) natif avec extension pg_trgm.
Conséquences	Indexation lexicale pondérée (setweight sur titre et légumes), lemmatisation française, tolérance aux fautes d'orthographe sans ajouter de conteneur lourd.

On a déjà PostgreSQL et on sait l'interroger via Doctrine, donc la recherche s'ajoute à la base existante sans nouveau conteneur ni outil à apprendre. Elle gère le français et les fautes de frappe, ce qui suffit pour des recettes.

LES JARDINS DE COCAGNE

4.4 RECHERCHE TEXTUELLE : POSTGRESQL FULL-TEXT SEARCH (FTS)

Indexation plein texte native :

- Dictionnaire french : lemmatisation des mots et suppression des mots vides.
- Colonne générée search_vector pondérée : priorité maximale sur le titre (A) et les légumes (B) face aux instructions (C).
- Tolérance aux fautes d'orthographe (Extension pg_trgm)
- Calcul de similarité trigramme permettant d'intercepter les erreurs de frappe (ex. frappe de « courget » qui propose immédiatement « courgette »).

LES JARDINS DE COCAGNE

ADR 09 : RECHERCHE SÉMANTIQUE ET SIMILARITÉ CULINAIRE

ADR 09 – Recherche sémantique

Contexte	Identifier des recettes équivalentes sans correspondance textuelle exacte (ex. « potée aux choux » et « mijoté de légumes d'hiver »), notamment pour valoriser des légumes oubliés du panier comme le rutabaga ou la blette.
Alternatives analysées	API externe OpenAI Embeddings : coût récurrent, dépendance à un tiers et fuite de données des adhérents. Base vectorielle dédiée Qdrant ou Milvus : conteneur lourd supplémentaire et administration complexe.
Décision	Modèle Hugging Face local (bge-small-fr) et stockage vectoriel via l'extension pgvector.
Conséquences	Données 100 % souveraines, zéro abonnement externe. Le modèle produit des vecteurs légers de dimension 384, calculés sur simple CPU. L'indexation HNSW de pgvector permet des calculs de similarité cosinus en moins de 5 ms, directement dans la base PostgreSQL déjà utilisée pour les données métiers et PostGIS.

On a besoin de proposer des recettes équivalentes même si les mots sont différents. Plutôt que de payer une API externe comme OpenAI qui ferait fuiter les données, ou d'alourdir Docker avec une base vectorielle dédiée (Qdrant), on fait tourner un modèle français léger (bge-small-fr) directement sur le serveur. Les vecteurs sont stockés dans notre PostgreSQL grâce à pgvector, ce qui garantit des suggestions en moins de 5 ms, une souveraineté totale des données et zéro frais d'abonnement

LES JARDINS DE COCAGNE

4.5 RECHERCHE SÉMANTIQUE : MODÉLISATION DES VECTEURS (PGVECTOR)

L’empreinte sémantique (vecteur de dimension 384) :

- Le texte de la recette (titre, légumes, cuisson) est lu par un modèle d'IA local (bge-small-fr).
- Il lui attribue une « adresse » sous forme de 384 nombres : chaque valeur capte une nuance culinaire (sucré/salé, soupe/plat au four, légumes racines/feuilles, temps de cuisson).
- Deux plats proches (ex. velouté de potimarron et soupe de butternut) obtiennent des coordonnées voisines.

L’index HNSW (Hierarchical Navigable Small World) :

- Structure en graphe multicouche qui navigue de groupe en groupe.
- Trouve les 5 recettes les plus proches parmi des milliers en moins de 5 ms, sans scanner toute la table.



LES JARDINS DE COCAGNE

4.6 DÉTECTION DE SIMILARITÉ & CAS D'USAGE

Alimentation automatique (Symfony) :

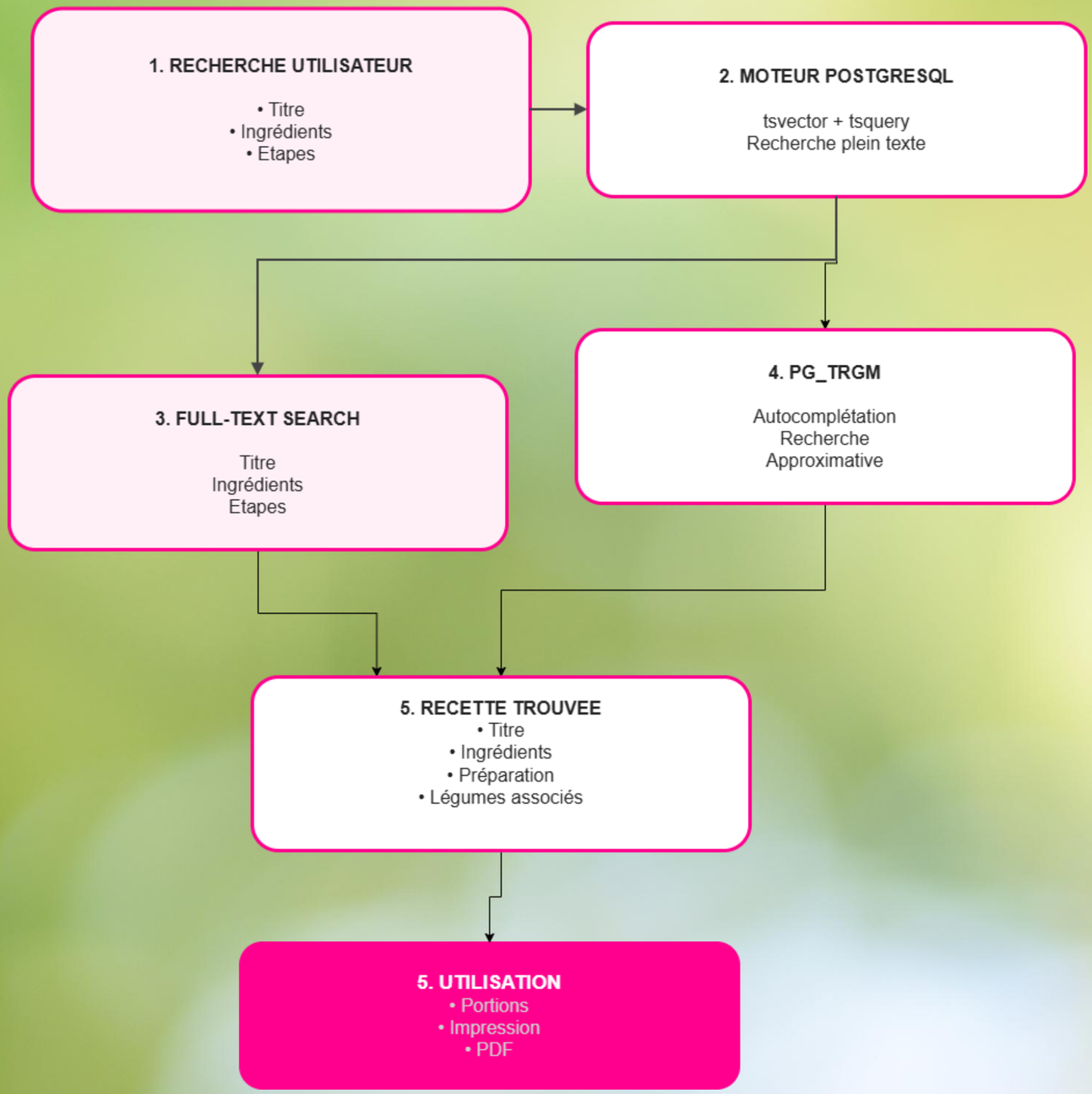
- À la création ou modification d'une recette sur le back-office, un événement RecetteSavedEvent génère automatiquement l'empreinte et met à jour la colonne embedding en base.

Valorisation côté adhérent :

- Variantes anti-gaspillage : suggestion automatique de plats aux profils proches pour écouler les surplus de légumes du panier.
- Remplacement immédiat : proposition d'une alternative culinaire si un légume de saison vient à manquer.

LES JARDINS DE COCAGNE

Gestion des recettes



PARTIE V
INDUSTRIALISATION, DÉPLOIEMENT & BILAN

LES JARDINS DE COCAGNE

1. CONTENEURISATION DOCKER

Architecture Docker Compose

Conteneur Web:
Serveur NGINX en frontal pour la gestion du trafic HTTP et la distribution rapide des fichiers statiques.

Conteneur App:
PHP 8.3-FPM préconfiguré avec les extensions requises (voir ci-contre).

Conteneur BDD:
PostgreSQL 16 officiel avec volume persistant sécurisé pour la sauvegarde des données.

Reproductibilité & Fiabilité

Environnement ISO:
Parité stricte entre les postes de développement des membres du binôme et le serveur de soutenance.

Déploiement en une commande:
`docker compose up -d` instancie et relie automatiquement l'ensemble des services.

Conteneur App (PHP 8.3-FPM)

- pdo_pgsql & pgsql = Communication entre Doctrine ORM et PostgreSQL
- opcache = Mise en cache mémoire du bytecode PHP (latence sous 200 ms)
- intl = Validation Symfony, gestion des dates de tournées et jours fériés
- zip = Gestion des dépendances Composer et exports documentaires
- ctype & iconv = Dépendances natives du framework Symfony et encodage UTF-8
- mbstring = Traitement des chaînes multi-octets et accents (recettes, dépôts)
- curl = Appels HTTP asynchrones vers l'API des jours fériés et le routeur OSRM

Conteneur BDD (PostgreSQL 16)

- postgis = Types géospatiaux GEOMETRY, index GIST, requêtes de proximité ST_DWithin
- pg_trgm = Recherche floue par trigrammes, tolérance aux fautes et autocomplétion
- unaccent = Suppression des accents pour la recherche plein texte (FTS)
- pgvector = Stockage des vecteurs (dimension 384) et index HNSW pour la recherche sémantique
- uuid-ossp (ou pgcrypto) = Génération native d'identifiants uniques universels (UUIDv4)

5.1 ARCHITECTURE CONTENEURISÉE DOCKER

Découpage des services :

- Service 'php' : Image personnalisée PHP 8.3-cli-alpine intégrant le serveur intégré pour Symfony, l'autoloader Composer et les extensions requises (pdo_pgsql, intl, opcache), accessible sur le port hôte 8090.
- Service 'database' : Image officielle PostgreSQL 16 Alpine avec volume persistant ('database_data') et vérification de santé ('healthcheck'), accessible en local sur le port 5432.
- Service 'db_admin' : Interface web d'administration de bases de données (Adminer / pgAdmin 4) pour la gestion visuelle du schéma, exposée sur le port 8091.
- Sécurité : Communication inter-conteneurs orchestrée via le réseau Docker par défaut, le service 'php' communiquant directement avec la base de données via l'hôte interne 'database'.

5.2 MIGRATIONS & FIXTURES REPRODUCTIBLES

Gestion des évolutions du schéma :

Versionnement systématique des schémas via le composant Doctrine Migrations.

Jeux de tests cohérents :

- 1 Structure pilote : Jardins de Thaon-les-Vosges.
- 14 Points de dépôts réels géo-référencés dans le bassin vosgien.
- Flotte de 3 camionnettes utilitaires (capacités de 40, 60 et 85 paniers).
- 250 Comptes adhérents avec abonnements paniers actifs pour simuler la charge réelle.

5.3 ASSURANCE QUALITÉ & MATRICE DE TESTS

Pyramide de tests :

- Tests unitaires (PHPUnit 11) : Couverture complète des algorithmes d'optimisation (Clarke & Wright, 2-opt).
- Tests d'intégration : Validation des transactions SQL, filtres multi-structures et Voters de sécurité.

Analyse statique :

- Exécution de PHPStan au niveau 8 pour garantir l'absence d'erreurs de typage.
- Conformité du code : Formatage automatisé respectant la norme PSR-12 via php-cs-fixer.

LES JARDINS DE COCAGNE

5.4 DOCKERFILE SINGLE-STAGE OPTIMISÉ

Construction de l'image de développement (Single-stage) :

- Image de base : php:8.3-cli-alpine configurée pour le développement local.
- Installation des dépendances système et bibliothèques C requises (icu-dev, libpq-dev, git, unzip).
- Compilation des extensions PHP nécessaires à Symfony et PostgreSQL (pdo_pgsql, intl, opcache).
- Nettoyage du cache du gestionnaire de paquets (apk --no-cache) pour limiter la taille de l'image.

On démare le serveur web interne de PHP pointant sur le dossier public/.
En développement une approche single-stage avec montage de volume hôte est preferable car elle conserve les outils CLI nécessaires au travail collaboratif et permet la modification du code à chaud sans reconstruction.
On a donc un environnement léger, autonome et synchronisé en direct avec les fichiers sources de la machine hôte.

LES JARDINS DE COCAGNE

COUVERTURE DES COMPÉTENCES ET CONCLUSION

Code Compétence	Libellé de l'Apprentissage Critique	Preuve d'Implémentation dans la SAÉ 5
AC31.01	Choisir et implémenter les architectures	Modèle 3-tiers modulaire, isolation multi-tenancy.
AC31.03	Intégrer en environnement de production	Docker Compose multi-stage, reverse proxy NGINX.
AC32.01	Anticiper les métriques et montées en charge	Modélisation CVRP, analyse combinatoire, index GIN.
AC32.03	Utiliser bibliothèques et méthodes dédiées	Heuristique Clarke & Wright, 2-opt, PostGIS et FTS.
AC36.01	Partager une veille technologique	Justification rigoureuse formalisée par 9 fiches ADR.

Les fiches ADR seront visibles dons le dossier document/fiche_adr du projet